Data Structure Laboratory with C

1	Design and implement a C program that dynamically manages an array using **dynamic memory allocation**. The program should allow the user to: a. **Insert** elements into the dynamically allocated array. b. **Delete** an element from the array while maintaining order. c. **Traverse** and display the elements of the array. d. **Search** for a specific element and display its position if found. The program should use malloc() or realloc() for memory management and properly free allocated memory after execution.

```
#include <stdio.h>
#include <stdlib.h>

int *arr = NULL;
int size = 0;

void insert(int data) {
   int *temp = (int*)realloc(arr, (size + 1) * sizeof(int));
   if (temp == NULL) {
      printf("ERROR: Reallocation failed.\n");
      return;
   }
   arr = temp;
   arr[size++] = data;
   printf("SUCCESS: %d inserted.\n", data);
}

void delete(int data) {
   if (size == 0) {
      printf("ERROR: Array empty.\n");
      return;
   }
   int i, j = -1;
   for (i = 0; i < size; i++) {
      if (arr[i] == data) {
         j = i;
         break;
      }
   }
   if (j == -1) {
      printf("ERROR: %d not found.\n", data);
      return;
```

```
    }
    for (i = j; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }
    size--;
    arr = (int*)realloc(arr, size * sizeof(int));
    printf("SUCCESS: %d deleted.\n", data);
}

void display() {
    if (size == 0) {
        printf("Array: [ Empty ]\n");
        return;
    }
    printf("Array: [ ");
    for (int i = 0; i < size; i++) {
        printf("%d%s", arr[i], (i == size - 1) ? "" : ", ");
    }
    printf(" ]\n");
}

void search(int data) {
    if (size == 0) {
        printf("Array empty. Cannot search.\n");
        return;
    }
    for (int i = 0; i < size; i++) {
        if (arr[i] == data) {
            printf("SUCCESS: %d found at index %d.\n", data, i);
            return;
        }
    }
    printf("Element %d not found.\n", data);
}

int main() {
    int choice, data;

    while (1) {
        printf("\nArray Menu\n1. Insert\n2. Delete\n3. Traverse\n4. Search\n5.
Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n');
```

```
            continue;
        }

        switch (choice) {
            case 1:
                printf("Enter element: ");
                scanf("%d", &data);
                insert(data);
                break;
            case 2:
                printf("Enter element to delete: ");
                scanf("%d", &data);
                delete(data);
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Enter element to search: ");
                scanf("%d", &data);
                search(data);
                break;
            case 5:
                if (arr != NULL) free(arr);
                printf("\nExiting. Memory freed.\n");
                return 0;
            default:
                printf("Invalid choice.\n");
        }
    }
}
```

Or

```
#include <stdio.h>
#include <stdlib.h> // Essential for malloc(), realloc(), and free()

int* arr = NULL; // Pointer to the dynamically allocated array
int size = 0;    // Current number of elements in the array

// Function Prototypes
void display_menu();
```

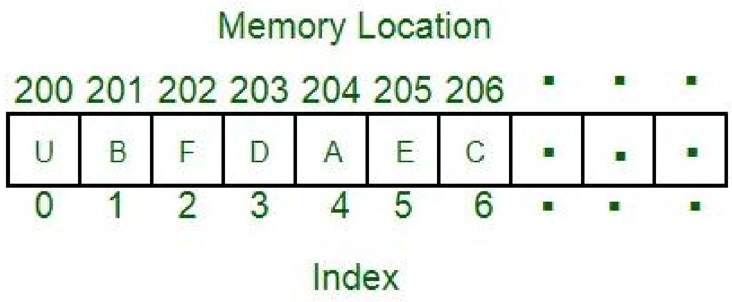
```
void insert_element(int data);
void delete_element(int data);
void traverse_array();
void search_element(int data);
void cleanup_memory();

int main() {
    int choice, data;

    while (1) {
        display_menu();
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("\nInvalid input. Please enter a number.\n");
            while (getchar() != '\n');
            continue;
        }

        switch (choice) {
            case 1:
                printf("Enter element to insert: ");
                scanf("%d", &data);
                insert_element(data);
                break;
            case 2:
                printf("Enter element to delete: ");
                scanf("%d", &data);
                delete_element(data);
                break;
            case 3:
                traverse_array();
                break;
            case 4:
                printf("Enter element to search: ");
                scanf("%d", &data);
                search_element(data);
                break;
            case 5:
                cleanup_memory(); // Call free() to release memory
                printf("\nExiting program. Memory freed.\n");
                return 0;
            default:
                printf("\nInvalid choice. Please try again.\n");
```

```
        }
    }
}

void display_menu() {
    printf("\nArray Menu\n");
    printf("1. Insert Element\n");
    printf("2. Delete Element\n");
    printf("3. Traverse/Display Array\n");
    printf("4. Search Element\n");
    printf("5. Exit\n");
}

void insert_element(int data) {
    size++; // Increment array size

    // Reallocate memory for the new size (size * sizeof(int))
    int *temp = (int*)realloc(arr, size * sizeof(int));

    if (temp == NULL) {
        printf("ERROR: Memory reallocation failed.\n");
        size--; // Revert size on failure
        return;
    }

    arr = temp; // Update array pointer

    arr[size - 1] = data;
    printf("SUCCESS: Element %d inserted.\n", data);
}

void delete_element(int data) {
    if (size == 0) return;

    int i, found_index = -1;
    // Find the element
    for (i = 0; i < size; i++) {
        if (arr[i] == data) {
            found_index = i;
            break;
        }
    }
```

```c
    if (found_index == -1) return;

    // Shift elements left to maintain order
    for (i = found_index; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }

    size--; // Decrement size

    // Shrink memory using realloc
    int *temp = (int*)realloc(arr, size * sizeof(int));

    // Update pointer, handling the case where realloc fails on shrink
    if (size == 0 || temp != NULL) {
        arr = temp;
    } else {
        printf("WARNING: Memory shrink failed.\n");
    }

    printf("SUCCESS: Element %d deleted.\n", data);
}

void traverse_array() {
    if (size == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("\nArray elements (%d total):\n[ ", size);
    for (int i = 0; i < size; i++) {
        printf("%d%s", arr[i], (i == size - 1) ? "" : ", ");
    }
    printf(" ]\n");
}

void search_element(int data) {
    if (size == 0) return;

    // Linear search
    for (int i = 0; i < size; i++) {
        if (arr[i] == data) {
            printf("SUCCESS: Element %d found at position (index) %d.\n", data, i);
            return;
        }
```

```
    }
    printf("Element %d not found in the array.\n", data);
}

void cleanup_memory() {
    if (arr != NULL) {
        free(arr);  // Release allocated memory
        arr = NULL; // Prevent a dangling pointer
    }
    size = 0;
}
```

Questions:
1. What is an array?
An array is a **collection of fixed number of homogeneous (same-type) data elements** stored in **contiguous memory locations**. It allows you to store multiple values of the same type under a single name.



2.How do you Declare an Array in C?

An array declaration requires specifying the **data type** of the elements and the **size** (number of elements).

Syntax:

dataType arrayName[size];

Example: Declaring an array of 10 integers.
int scores[10];

3.How do you Initialize an Array?

You can initialize an array at the time of declaration in several ways:

int nums[5] = {10, 20, 30, 40, 50};The compiler knows the size is **5**.
int nums[5] = {10, 20};Remaining elements (3rd, 4th, 5th) are **automatically initialized to 0** (Zero).
int nums[] = {10, 20, 30};The compiler automatically calculates the size as **3** based on the number of initializers.

4.How do you Assign/Access Values in an Array?

Array elements are accessed using the array name followed by the index in square brackets.

Example:

```
int marks[3]; // Declare an array of size 3

// Assigning values:
marks[0] = 95; // Assign 95 to the 1st element (index 0)
marks[1] = 88;
marks[2] = 72;

// Accessing and printing the 2nd element:
printf("The second mark is: %d\n", marks[1]); // Output: 88

// Assigning a new value to the last element:
marks[2] = marks[2] + 5; // New value for marks[2] is 77
```

5.Dynamic Memory Allocation (DMA)

The process of allocating memory manually during the **run-time** of a program, typically on the **Heap** section of memory, using library functions like malloc(), calloc(), realloc(), and free().

6.Where is dynamically allocated memory stored?

The **Heap**. (Contrast with static/local variables, which go on the Stack or Data/BSS segments).

7.What is the return type of malloc()?

void * (a generic pointer). It must be explicitly **type-casted** to the required pointer type.

8.malloc() vs. realloc()

Function	Purpose	Size Argument	Initializes Memory
malloc()	Allocates a block of memory of the specified **size**.	Takes **one** argument: size in bytes.	Does **not** initialize (contains garbage values).
realloc()	Resizes a previously allocated block of memory.	Takes **two** arguments: pointer to the block and the new size in bytes.	Preserves the content of the old memory block up to the minimum of the new and old sizes.

9.How does realloc() handle resizing?

If there's adjacent free memory, it may extend the block in place. If not, it allocates a new, larger block, copies the data, and **frees the old block**. It returns a pointer to the *new* block

free()

Deallocates the memory block pointed to by the given pointer, making it available for future allocations.

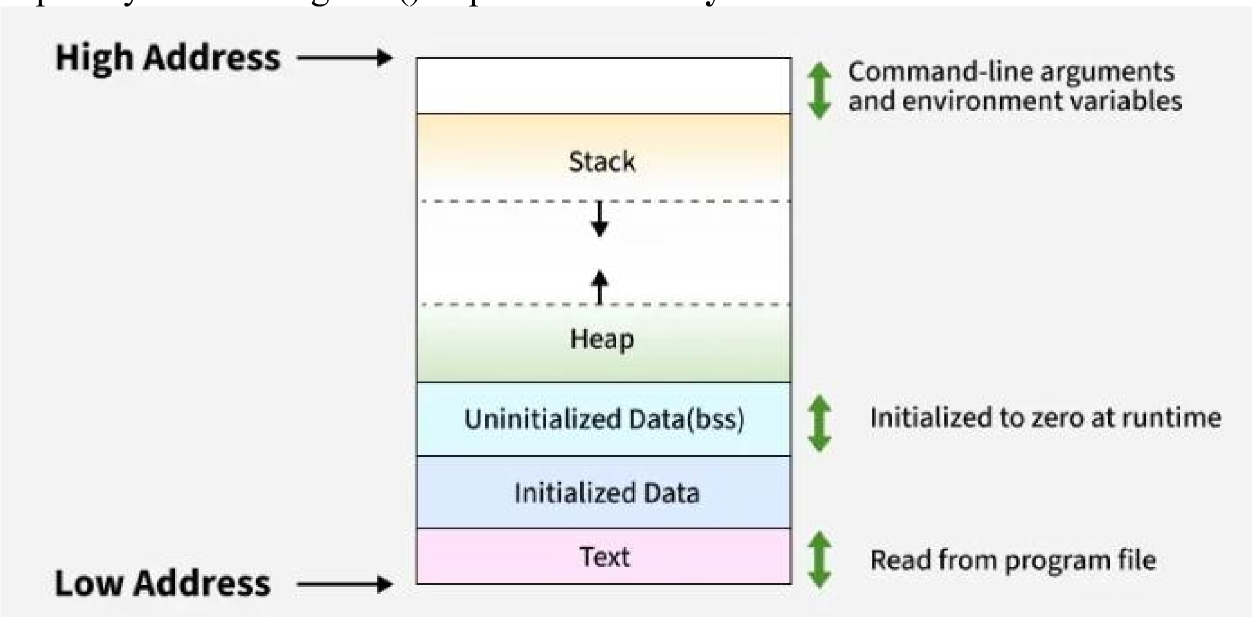
10.What is a "Dangling Pointer"?

A pointer that points to a memory location that has been deallocated (freed). Accessing memory via a dangling pointer leads to **undefined behavior** (a common runtime error).

11.Heap

The **Heap** is a large pool of memory used for **Dynamic Memory Allocation (DMA)**. It's managed by the operating system and is where memory is allocated during program **runtime** using functions like malloc() and realloc().

Memory on the Heap is **not automatically managed**; the programmer must explicitly free it using free() to prevent **memory leaks**.



C Program Memory Layout

12.Type Casting

Type Casting is the process of converting a value from one data type to another.

It tells the compiler to treat a variable or expression as if it were a different data type.

Types:

Implicit: Done automatically by the compiler (e.g., converting an int to a float).

Explicit: Done manually by the programmer using the cast operator (type). This is essential for DMA.

Simple Example (Explicit Casting)

When using malloc(), we must explicitly cast the generic void * pointer it returns to the specific type we need:

C
// Without type casting, the compiler might warn/error:// int *ptr = malloc(10 * sizeof(int));
// WITH explicit type casting:int *ptr = (int *)malloc(10 * sizeof(int));// The (int *) cast converts the generic void* to an int* pointer.

13.Dynamic Memory Allocation Functions

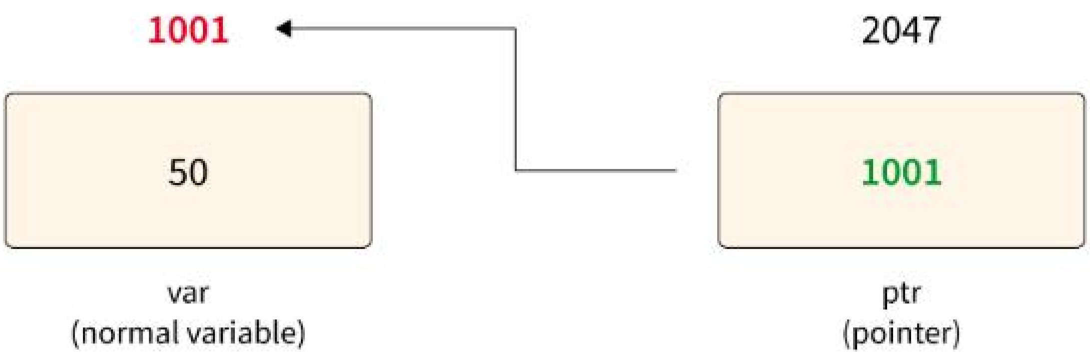
These functions are used to manage the Heap memory.

Function	Simple Definition	Usage Example
malloc()	**M**emory **Alloc**ation. Allocates a block of requested size in bytes.	ptr = malloc(size_of_block);
calloc()	**C**ontiguous **Alloc**ation. Allocates space for an array of elements, and **initializes** all bytes to **zero**.	ptr = calloc(num_elements, element_size);
realloc()	**Re**allocate memory. Changes (resizes) the size of a previously allocated memory block.	new_ptr = realloc(old_ptr, new_size);
free()	Deallocates (releases) the memory block previously allocated by malloc(), calloc(), or realloc().	free(ptr);

14.What is a Pointer?

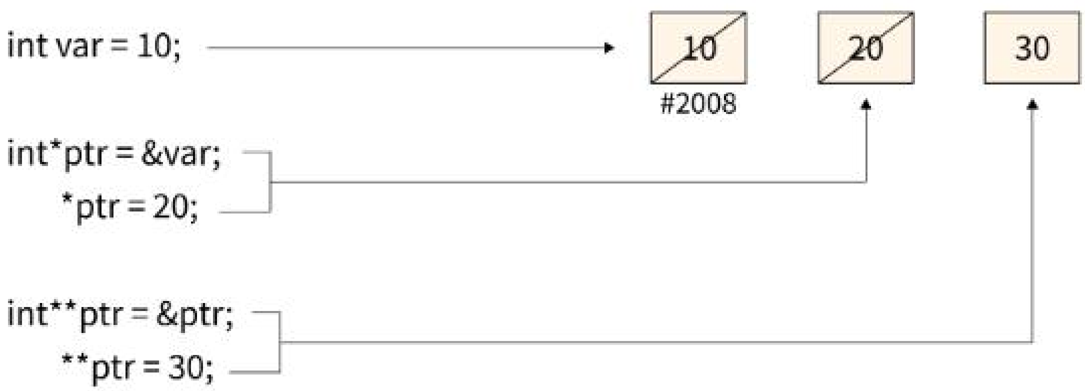
A pointer is a special variable that is used to store the memory address of another variable (which is of the same data type as the pointer) as its value rather than storing the direct values. A pointer variable stores the value of another variable of the same data type, for example, if the variable is of **int** data type, then the pointer which will store the memory address of this variable is also of the **int** data type

In C language, a pointer can be declared using the * (asterisk) operator. By using this operator, the address of the variable (on which we are working) is assigned to the pointer variable.



15.What is Dereferencing?

Dereferencing is used to access or find out the data which is contained in the memory location which is pointed by the pointer. The * (asterisk) operator which is also known as the C dereference pointer is used with the pointer variable to dereference the pointer. When we dereference a pointer, the data of the variable which is pointed by this pointer is returned.



16.How to Dereference a Pointer?

The dereferencing of a pointer is done by using the * (asterisk) symbol in front of the pointer making it a C dereference pointer.

```
int main() {
  int x = 19,deref;
  int *ptr;        // creating a pointer
  ptr = &x;        // Stores address of x in ptr
  deref = *ptr;    // Put Value at ptr in deref
  printf("%d",deref);
}
```

Output

19

Creating a C Dereference Pointer of Character Data Type

```
int main() {
   char ch = 'a',deref;
   char *ptr;      // creating a pointer
   ptr = &ch;      // Stores address of ch in ptr
   deref = *ptr;   // Put Value at ptr in deref
   printf("%c",deref);
}
```

Output

a

17.What is int* arr?

int* arr is a **variable declaration in C (and C++)** that defines arr as a **pointer to an integer**.

int**Data Type**Specifies the type of data the pointer will point to (in this case, an integer).***Dereference Operator (in this context, Declaration)**Indicates that arr is a **pointer** variable, meaning it stores a memory address, not a value.arr**Variable Name**The identifier used to refer to the pointer variable.

arr stores an address: arr doesn't store an actual integer value (like 10 or 20). Instead, it stores the **memory address** (location) where an integer is kept.

18.The functions required for allocating, resizing, and deallocating memory on the **Heap** are defined here:

malloc(): Allocates a block of uninitialized memory.

calloc(): Allocates memory for an array and initializes it to zero.

realloc(): Resizes a previously allocated memory block (crucial for your dynamic array).

free(): Deallocates memory, preventing memory leaks.